

MetroSim

Симулатор на транспортна мрежа с паралелна обработка
Документация на проекта

Алекс Цветанов, 121222225, КСИ, ФКСТ
Кирил Вълков, 121222194, КСИ, ФКСТ

13 октомври 2025 г.

Съдържание

1	Въведение	4
1.1	Обща информация за проекта	4
1.2	Ключови характеристики	4
1.3	Технически изисквания	4
2	Идея и концепция на приложението	4
2.1	Основна идея	4
2.2	Симулационен модел	4
2.3	Визуализация и взаимодействие	5
3	Архитектура на системата	5
3.1	Слоеста архитектура	5
3.2	Модел на паралелност	5
4	Детайлен анализ на класовете	5
4.1	Domain слой	5
4.1.1	Класове и интерфейси	5
4.2	Infrastructure слой	6
4.2.1	SimulationEngine	6
4.2.2	SimulationFacade	6
4.3	Services слой	7
4.3.1	Routing подсистема	7
4.4	UI слой	7
4.4.1	MainWindow	7
5	Шаблони за проектиране (Design Patterns)	7
5.1	Създаващи шаблони (Creational Patterns)	7
5.1.1	Abstract Factory	7
5.1.2	Builder	8
5.2	Структурни шаблони (Structural Patterns)	8
5.2.1	Facade	8
5.2.2	Decorator	9
5.2.3	Adapter	9
5.3	Поведенчески шаблони (Behavioral Patterns)	10
5.3.1	Observer	10
5.3.2	State	10
5.3.3	Strategy	11
5.3.4	Command + Memento	11
6	Конкурентност и паралелизъм	12
6.1	Модел на нишки	12
6.2	Архитектура на нишките	12
6.3	Синхронизационни механизми	12
6.3.1	CancellationToken	12
6.3.2	Неизменяеми обекти (Immutable Objects)	13
6.4	Предотвратяване на състезателни условия	13
7	Метрики и наблюдение	13
7.1	MetricsRegistry система	13
7.2	Типове метрики	13
7.3	Декоратори за метрики	13
8	Система за плъгини и разширяемост	14

8.1	ServiceLoader архитектура	14
8.2	Конфигурационен файл	14
8.3	Примерни плъгини	14
8.3.1	Sofa Pack Factory	14
8.4	Точки за разширение	14
9	Тестване и качество на кода	15
9.1	Архитектурни принципи	15
9.2	Кодови конвенции	15
9.3	Паралелно тестване	15
10	Инструкции за стартиране и конфигуриране	15
10.1	Системни изисквания	15
10.2	Компилиране и стартиране	15
10.2.1	Основни команди	15
10.2.2	Конфигурационни опции	15
10.3	Структура на проекта	16
11	Бъдещо развитие и разширения	17
11.1	Планирани функции	17
11.2	Възможни разширения	18
11.2.1	Нови типове превозни средства	18
11.2.2	Сложни маршрути	18
11.2.3	Интерактивност	18
12	Заклучение	18
12.1	Постигнати цели	18
12.2	Образователна стойност	18
12.3	Техническо качество	18
12.4	Практическа приложимост	19

1 Въведение

1.1 Обща информация за проекта

MetroSim е десктоп симулатор на транспортна мрежа, който визуализира движението на превозни средства в опростена метро/автобусна система. Проектът е изграден с Java 17 и използва Swing за графичния интерфейс.

Основната цел на приложението е да демонстрира как множество шаблони за проектиране (Design Patterns) могат да бъдат комбинирани в паралелно, събитийно и разширяемо приложение. Това го прави отлична примерна имплементация за университетски курсове по:

- Шаблони за проектиране – демонстрация на практическото приложение на GoF шаблоните
- Паралелно програмиране – безопасна работа с нишки в графични приложения
- Софтуерна архитектура – изграждане на модулни и разширяеми системи

1.2 Ключови характеристики

- Реално време симулация на превозни средства и световен часовник
- Отзивчив Swing интерфейс – актуализации чрез неизменяеми WorldSnapshot обекти
- Безопасен паралелизъм използвайки ForkJoinPool и ScheduledExecutorService
- Разширяем дизайн – нови маршрутизатори, градове и метрики чрез плъгини
- Command pattern каркас за бъдещи функции за редактиране/undo-redo

1.3 Технически изисквания

- Java Development Kit (JDK) 17 или по-нова версия
- Gradle за управление на зависимостите и билд процеса
- Операционна система: Windows, macOS, Linux с поддръжка на Java Swing

2 Идея и концепция на приложението

2.1 Основна идея

Основната идея зад MetroSim е да се създаде симулатор, който моделира реалистично поведение на транспортна мрежа. Всяко превозно средство в системата:

1. Започва от определен възел (позиция) в мрежата
2. Движи се стъпка по стъпка към крайната си дестинация
3. Спира за кратък престой („dwell“) при достигане на дестинацията
4. Обръща посоката и продължава към нова дестинация
5. Работи едновременно с всички останали превозни средства в системата

2.2 Симулационен модел

Симулацията се основава на дискретно време с фиксирани интервали (tick-ове). На всеки tick всички превозни средства актуализират своето състояние паралелно. Това позволява:

- Детерминистично поведение – резултатите са предвидими и възпроизводими
- Ефективна паралелизация – всяко превозно средство може да се обработва независимо
- Лесно тестване – състоянието може да се проверява на всеки времеви интервал

2.3 Визуализация и взаимодействие

Потребителският интерфейс предоставя:

- Карта – визуализация на движещите се превозни средства
- Контролни бутони – Start/Pause за управление на симулацията
- Панел за инспекция – детайлна информация за всяко превозно средство
- Времева линия – индикатор за напредъка на симулацията

3 Архитектура на системата

3.1 Слоеста архитектура

MetroSim използва слоеста архитектура с ясно разделение на отговорностите:

Слой	Описание
app/	Главна точка на стартиране и инициализация на компонентите
ui/	Графичен интерфейс (Swing) – прозорци, панели и медиатор
domain/	Основна симулационна логика – свят, превозни средства, състояния
infrastructure/	Конкурентен двигател и фасада за управление на симулацията
services/	Алгоритми за маршрутизиране, зареждане на сценарии и плъгини
metrics/	Система за измервания и статистики
themes/	Теми и цветови пакети чрез плъгини
events/	Събитийна система (Observer) за обновяване на интерфейса

Таблица 1: Слоеве в архитектурата на MetroSim

3.2 Модел на паралелност

Системата използва няколко типа нишки за различни задачи:

Тип нишка	Отговорност
EDT (Event Dispatch Thread)	Обработка на графиката и взаимодействие с потребителя
Симулационен планировчик	Задейства цикъла на симулацията на определени интервали
Работен пул (ForkJoinPool)	Изпълнява обновленията на превозните средства паралелно

Таблица 2: Модел на паралелност в MetroSim

4 Детайлен анализ на класовете

4.1 Domain слой

4.1.1 Класове и интерфейси

World Централният клас, който съхранява състоянието на цялата симулация:

```

1 public final class World {
2     private final List<Vehicle> vehicles = new ArrayList<>();
3     private final SimClock clock = new SimClock();
4
5     public List<Vehicle> vehicles() { return vehicles; }
6     public SimClock clock() { return clock; }
7     public void advanceClock() { clock.advance(1); }
8     public WorldSnapshot snapshot(long tickId) { ... }
9 }

```

Listing 1: Основни методи на класа World

Vehicle Представява превозно средство с вътрешни състояния:

- Idle – начално състояние
- EnRoute – движение към дестинация
- Dwell – престой на спирка

VehicleState Интерфейс за имплементация на State pattern:

```

1 public interface VehicleState {
2     void onTick(Vehicle ctx, SimClock clock, CancellationToken ct);
3     String name();
4 }

```

Listing 2: Интерфейс VehicleState

WorldSnapshot и VehicleSnapshot Неизменяеми обекти (records), които представляват състоянието в даден момент:

```

1 public record WorldSnapshot(long tickId, List<VehicleSnapshot> vehicles) {
2     public static final WorldSnapshot EMPTY = new WorldSnapshot(0, List.of());
3 }
4
5 public record VehicleSnapshot(String label, int nodeIndex, String stateName) {}

```

Listing 3: Snapshot класове

4.2 Infrastructure слой

4.2.1 SimulationEngine

Ядрото на симулацията, което обработва всички превозни средства паралелно:

```

1 public WorldSnapshot step(CancellationToken ct) {
2     List<Callable<Void>> tasks = new ArrayList<>();
3     for (Vehicle v : world.vehicles())
4         tasks.add(() -> {
5             v.tick(world.clock(), ct);
6             return null;
7         });
8     invokeAllCancellable(tasks, ct);
9     world.advanceClock();
10    return world.snapshot(tick.incrementAndGet());
11 }

```

Listing 4: Ключов метод на SimulationEngine

4.2.2 SimulationFacade

Фасада, която скрива сложността на управлението на нишки и синхронизация:

```

1 public synchronized void start() {
2     if (token.isCancelled()) {
3         token = new CancellationToken();
4     }
5     runner = scheduler.scheduleAtFixedRate(
6         () -> {
7             if (token.isCancelled()) return;
8             WorldSnapshot s = engine.step(token);
9             snapshots.publish(s);
10        }, 0, 50, TimeUnit.MILLISECONDS);
11 }

```

Listing 5: SimulationFacade управление

4.3 Services слой

4.3.1 Routing подсистема

Системата за маршрутизиране използва Strategy pattern с няколко имплементации:

- TimeOptimalRouter – базов алгоритъм за оптимално време
- ThirdPartyRouterAdapter – адаптира външни библиотеки
- LoggingRouter – декоратор за логване
- MeteredRouter – декоратор за метрики

4.4 UI слой

4.4.1 MainWindow

Главният прозорец на приложението, който координира всички панели:

```

1 public MainWindow(Bootstrap.Components c) {
2     MapPanel map = new MapPanel(c.cityPack);
3     TimelinePanel timeline = new TimelinePanel();
4     InspectorPanel inspector = new InspectorPanel();
5
6     c.simulation.snapshots().subscribe(map);
7     c.simulation.snapshots().subscribe(timeline);
8     c.simulation.snapshots().subscribe(inspector);
9
10    new MainWindowMediator(c.simulation, start, pause);
11 }

```

Listing 6: Структура на MainWindow

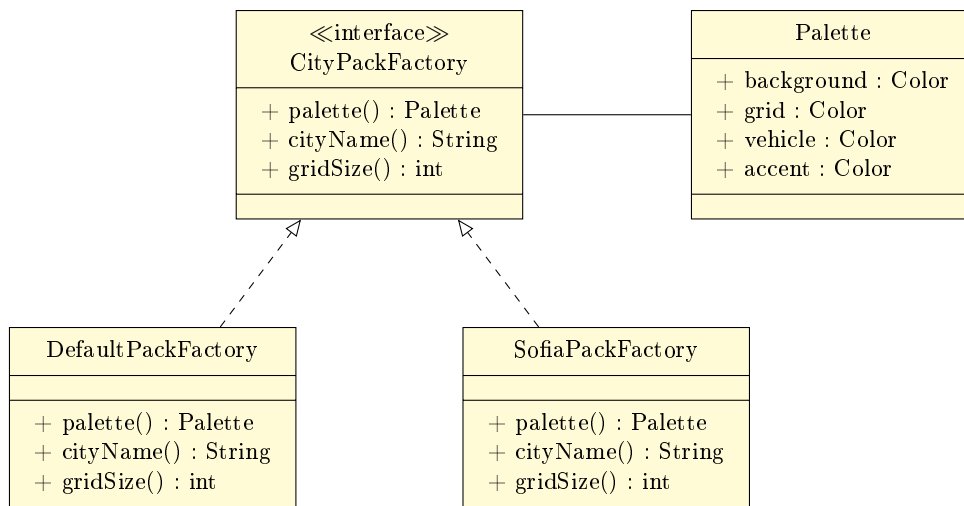
5 Шаблони за проектиране (Design Patterns)

5.1 Създаващи шаблони (Creational Patterns)

5.1.1 Abstract Factory

Цел: Предоставя интерфейс за създаване на семейства от свързани обекти.

Имплементация: Използва се за създаване на различни теми (Default, Sofia).



Фигура 1: UML диаграма на Abstract Factory pattern

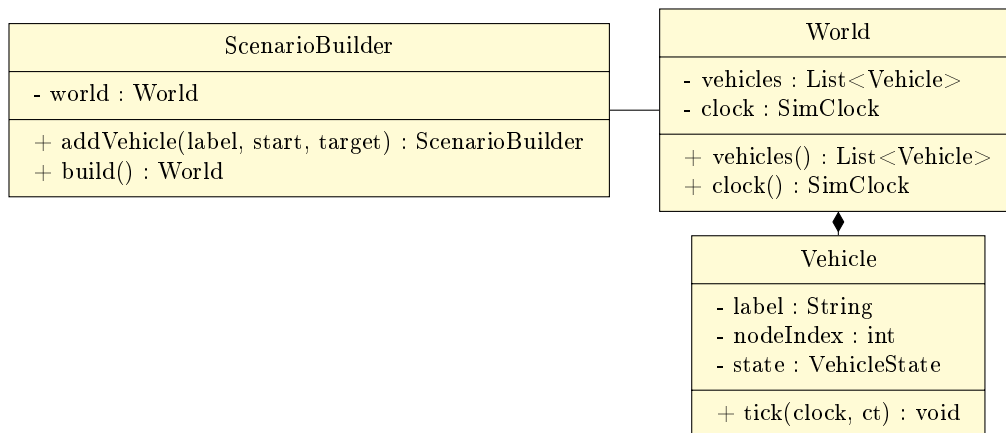
Предимства:

- Лесно добавяне на нови теми без промяна на съществуващия код
- Гарантира съвместимост между компонентите на дадена тема
- Поддържа плагин архитектура чрез ServiceLoader

5.1.2 Builder

Цел: Позволява създаването на сложни обекти стъпка по стъпка.

Имплементация: ScenarioBuilder изгражда конфигурации на света.



Фигура 2: UML диаграма на Builder pattern

Предимства:

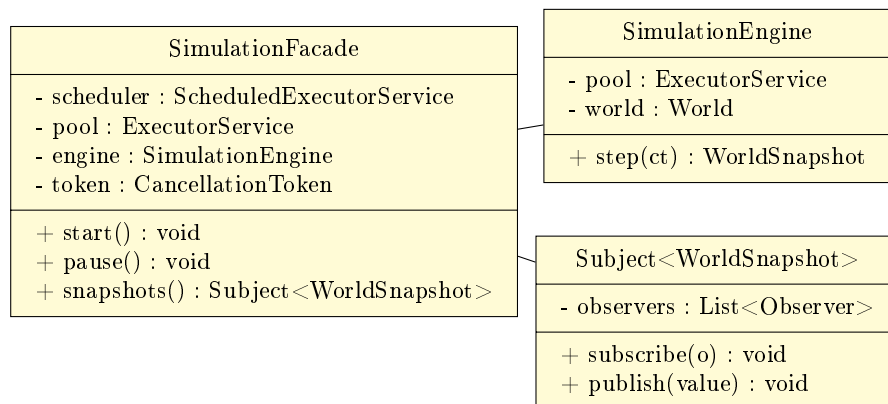
- Флуентен API за създаване на сценарии
- Възможност за различни конфигурации без множество конструктори
- Лесно тестване с различни настройки

5.2 Структурни шаблони (Structural Patterns)

5.2.1 Facade

Цел: Предоставя опростен интерфейс към сложна подсистема.

Имплементация: SimulationFacade скрива сложността на управлението на нишки.



Фигура 3: UML диаграма на Facade pattern

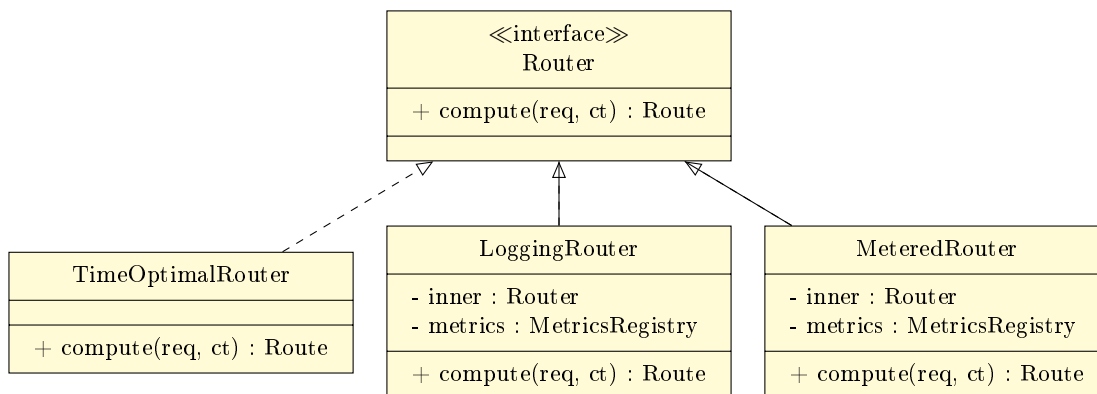
Предимства:

- Скрива сложността на паралелното изпълнение
- Предоставя прост интерфейс за клиентите
- Централизира управлението на ресурсите

5.2.2 Decorator

Цел: Добавя ново поведение към обекти динамично без промяна на тяхната структура.

Имплементация: MeteredRouter и LoggingRouter добавят функционалност към базовите маршрутизатори.



Фигура 4: UML диаграма на Decorator pattern

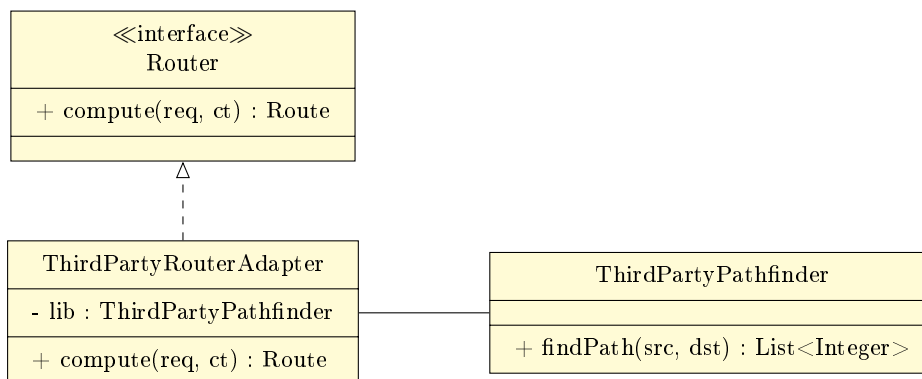
Предимства:

- Гъвкаво композиране на функционалности
- Спазване на Open/Closed principle
- Възможност за многократно декориране

5.2.3 Adapter

Цел: Позволява на класове с несъвместими интерфейси да работят заедно.

Имплементация: ThirdPartyRouterAdapter адаптира външна библиотека за маршрутизиране.



Фигура 5: UML диаграма на Adapter pattern

Предимства:

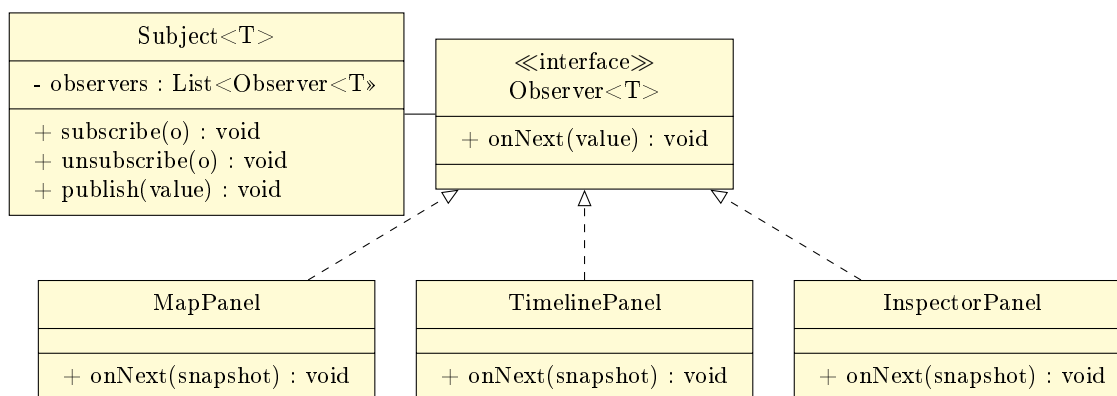
- Интеграция на външни библиотеки без промяна в тях
- Запазване на съществуващите интерфейси
- Изолиране на зависимостите към външни системи

5.3 Поведенчески шаблони (Behavioral Patterns)

5.3.1 Observer

Цел: Дефинира зависимост тип „един-към-много“ между обекти.

Имплементация: Subject публикува snapshot-и към UI панелите.



Фигура 6: UML диаграма на Observer pattern

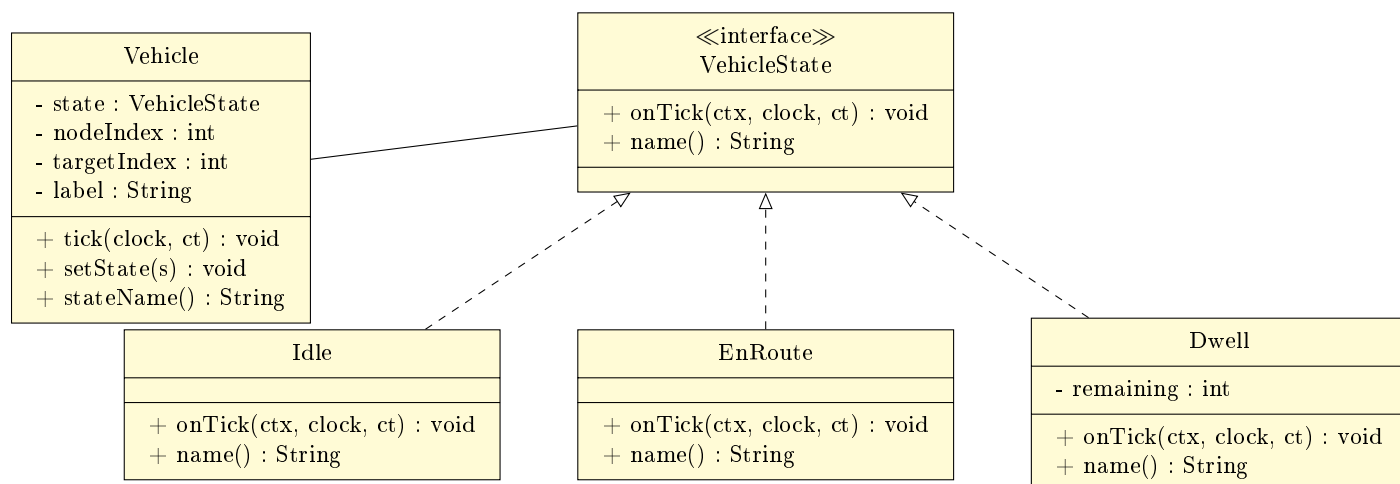
Предимства:

- Слабо свързване между издател и наблюдатели
- Динамично добавяне/премахване на наблюдатели
- Поддържа broadcast комуникация

5.3.2 State

Цел: Позволява на обект да променя поведението си при промяна на вътрешното състояние.

Имплементация: Vehicle преминава през различни състояния (Idle → EnRoute → Dwell).



Фигура 7: UML диаграма на State pattern

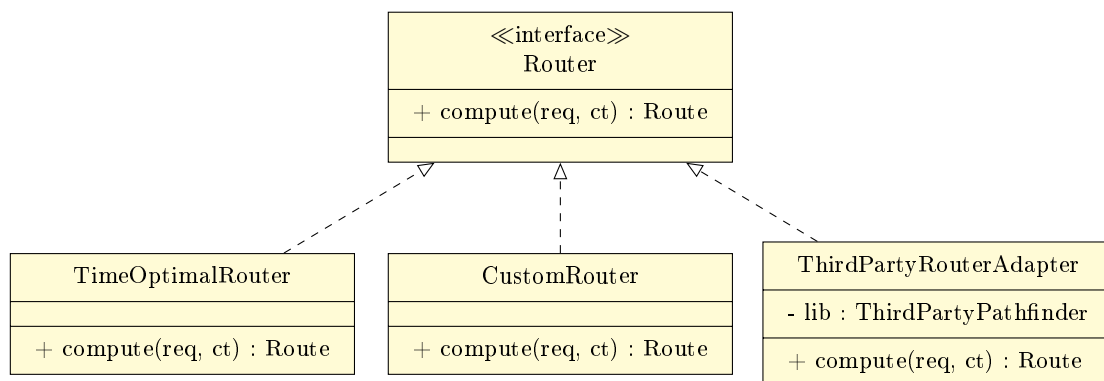
Логика на преходите:

- Idle → EnRoute: При започване на симулацията
- EnRoute → Dwell: При достигане на дестинация
- Dwell → EnRoute: След изтичане на времето за престой

5.3.3 Strategy

Цел: Дефинира семейство от алгоритми и ги прави взаимозаменяеми.

Имплементация: Router интерфейсът позволява различни алгоритми за маршрутизиране.



Фигура 8: UML диаграма на Strategy pattern

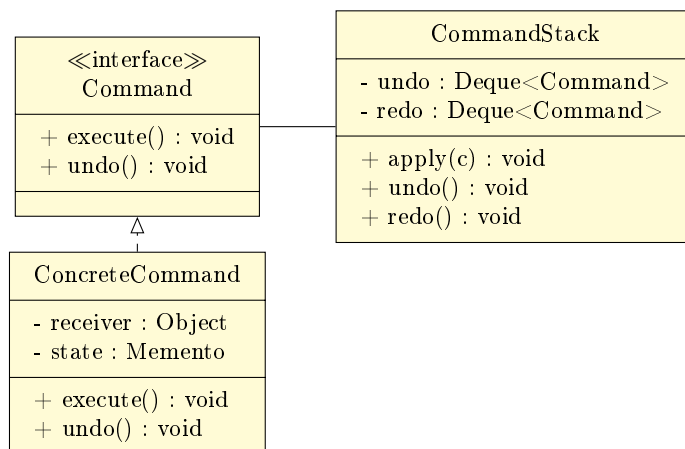
Стратегии:

- TimeOptimalRouter: Най-кратко време за пътуване
- ThirdPartyRouterAdapter: Използва външна библиотека
- CustomRouter: Потребителски алгоритми (възможност за разширение)

5.3.4 Command + Memento

Цел: Капсулира заявка като обект и поддържа undo/redo операции.

Имплементация: CommandStack осигурява scaffold за бъдещи editing функции.



Фигура 9: UML диаграма на Command + Memento pattern

Възможности за разширение:

- Команди за добавяне/премахване на превозни средства
- Команди за промяна на маршрути
- Команди за конфигуриране на симулацията

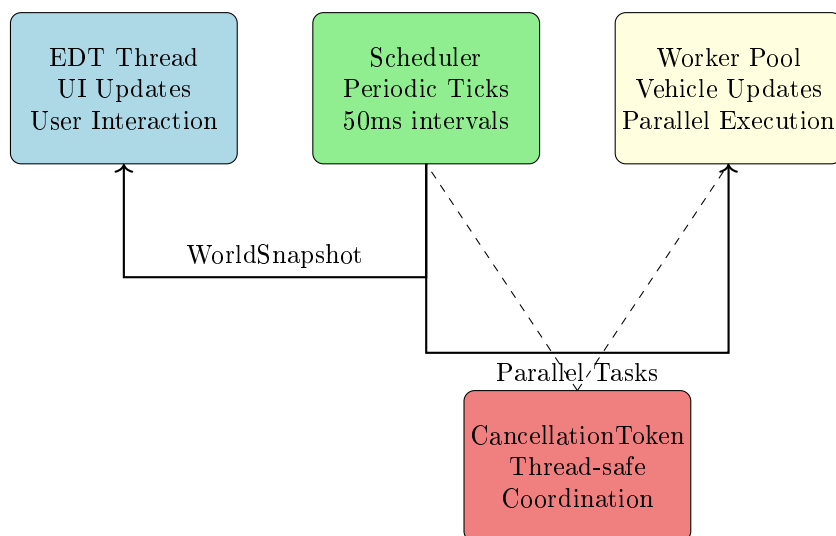
6 Конкурентност и паралелизъм

6.1 Модел на нишки

MetroSim използва софистициран модел на паралелност за да осигури:

- Отзивчив потребителски интерфейс
- Ефективна обработка на превозните средства
- Безопасна синхронизация

6.2 Архитектура на нишките



Фигура 10: Архитектура на нишките в MetroSim

6.3 Синхронизационни механизми

6.3.1 CancellationToken

Осигурява кооперативно прекъсване на операции:

```

1 public final class CancellationToken {
2     private final AtomicBoolean cancelled = new AtomicBoolean(false);
3
4     public boolean isCancelled() {
5         return cancelled.get();
6     }
7
8     public void cancel() {
9         cancelled.set(true);
10    }
11 }

```

Listing 7: CancellationToken имплементация

6.3.2 Неизменяеми обекти (Immutable Objects)

Всички данни, които се предават между нишки, са неизменяеми:

- WorldSnapshot – състояние на света
- VehicleSnapshot – състояние на превозно средство
- Route – маршрут между две точки

6.4 Предотвратяване на състезателни условия

1. Сору-on-read: Снимките се правят преди публикуване
2. Атомични операции: Използват се за ъпдейт на състоянието
3. Synchronized блокове: За критичните секции в SimulationFacade
4. Volatile променливи: За видимост между нишки

7 Метрики и наблюдение

7.1 MetricsRegistry система

Системата за метрики позволява проследяване на производителността:

```

1 public interface MetricsRegistry {
2     void counter(String name, long delta);
3     void recordTime(String name, long nanos);
4     long getCounter(String name);
5     long getTimeSum(String name);
6 }

```

Listing 8: MetricsRegistry интерфейс

7.2 Типове метрики

Тип метрика	Описание
routing.count	Общ брой извършени маршрутизации
routing.total.ns	Общо време за маршрутизация в наносекунди
routing.logged	Брой логнати маршрутизации
simulation.ticks	Общ брой симулационни tick-ове

Таблица 3: Поддържани метрики в системата

7.3 Декоратори за метрики

Метриките се събират чрез Decorator pattern:

```

1  public Route compute(RouteRequest req, CancellationToken ct) {
2      long t0 = System.nanoTime();
3      try {
4          return inner.compute(req, ct);
5      } finally {
6          metrics.recordTime("routing.total.ns", System.nanoTime() - t0);
7          metrics.counter("routing.count", 1);
8      }
9  }

```

Listing 9: MeteredRouter декоратор

8 Система за плъгини и разширяемост

8.1 ServiceLoader архитектура

MetroSim използва Java ServiceLoader за откриване на плъгини по време на изпълнение:

```

1  private static CityPackFactory resolveCityPack() {
2      String want = System.getProperty("metrosim.cityPack", "");
3      CityPackFactory fallback = new DefaultPackFactory();
4      var loader = ServiceLoader.load(CityPackFactory.class);
5
6      for (CityPackFactory f : loader) {
7          String name = f.cityName().toLowerCase();
8          if (!want.isEmpty() && name.contains(want)) {
9              return f;
10         }
11     }
12     return fallback;
13 }

```

Listing 10: Плъгин discovery в Bootstrap

8.2 Конфигурационен файл

Плъгините се регистрират в META-INF/services/metrosim.themes.CityPackFactory:

```

1  metrosim.themes.DefaultPackFactory
2  metrosim.themes.SofiaPackFactory

```

Listing 11: Регистрация на плъгини

8.3 Примерни плъгини

8.3.1 Sofia Pack Factory

```

1  public final class SofiaPackFactory implements CityPackFactory {
2      public Palette palette() {
3          return new Palette(
4              new Color(0x0f1220), // background
5              new Color(0x2d355c), // grid
6              new Color(0x58d68d), // vehicle
7              new Color(0xf7dc6f)  // accent
8          );
9      }
10
11     public String cityName() { return "Sofia"; }

```

```
12 public int gridSize() { return 22; }  
13 }
```

Listing 12: SofiaPackFactory имплементация

8.4 Точки за разширение

Системата предоставя следните точки за разширение:

- CityPackFactory – нови визуални теми
- Router – нови алгоритми за маршрутизиране
- VehicleState – нови поведения на превозните средства
- Command – нови команди за редактиране
- Observer – нови типове панели за визуализация

9 Тестване и качество на кода

9.1 Архитектурни принципи

Проектът следва основни принципи за качествен софтуер:

- SOLID принципи – всеки клас има ясна отговорност
- DRY (Don't Repeat Yourself) – избягване на дублиране на код
- KISS (Keep It Simple, Stupid) – простота в дизайна
- Separation of Concerns – ясно разделение на отговорностите

9.2 Кодови конвенции

Проектът използва:

- Google Java Format за форматиране на кода
- Spotless за автоматична корекция на стила
- Gradle за управление на build процеса

9.3 Паралелно тестване

Архитектурата позволява лесно unit тестване:

- Injection на зависимости чрез конструктори
- Интерфейси за всички основни компоненти
- Неизменяеми обекти за предвидимо поведение
- CancellationToken за контролирано прекъсване

10 Инструкции за стартиране и конфигуриране

10.1 Системни изисквания

- Java Development Kit (JDK) 17 или по-нова версия
- Операционна система с поддръжка на Java Swing
- Минимум 512MB RAM за изпълнение

10.2 Компилиране и стартиране

10.2.1 Основни команди

```

1  ./ gradlew build
2
3  ./ gradlew run
4
5  ./ gradlew run -Dmetrosim.cityPack=sofia
6  ./ gradlew clean

```

Listing 13: Основни Gradle команди

10.2.2 Конфигурационни опции

Системно свойство	Описание
metrosim.cityPack	Избор на градска тема (default, sofia)
java.awt.headless	Задава се на false за GUI режим

Таблица 4: Конфигурационни параметри

10.3 Структура на проекта

```

1  | build.gradle
2  | gradlew
3  | gradlew.bat
4  | README.md
5  | settings.gradle
6  |
7  | --- github
8  | | --- workflows
9  | |     gradle.yml
10 |
11 | --- documentation
12 | | MetroSim_Documentation_BG.aux
13 | | MetroSim_Documentation_BG.log
14 | | MetroSim_Documentation_BG.out
15 | | MetroSim_Documentation_BG.pdf
16 | | MetroSim_Documentation_BG.synctex.gz
17 | | MetroSim_Documentation_BG.tex
18 | | MetroSim_Documentation_BG.toc
19 | | tikz-uml.sty
20 |
21 | --- gradle
22 | | --- wrapper
23 | |     gradle-wrapper.jar
24 | |     gradle-wrapper.properties
25 |
26 | --- src
27 | | --- main
28 | | | --- java
29 | | | | --- metrosim
30 | | | |     --- app
31 | | | | | Bootstrap.java
32 | | | | | MetroSimApp.java
33 | | |
34 | | | --- domain
35 | | | | Passenger.java
36 | | | | SimClock.java

```



```

37         Vehicle.java
38         VehicleSnapshot.java
39         VehicleState.java
40         World.java
41         WorldSnapshot.java
42
43     --- events
44         Event.java
45         EventType.java
46         Observer.java
47         Subject.java
48
49     --- infrastructure
50         SimulationEngine.java
51         SimulationFacade.java
52
53     --- metrics
54         MetricsRegistry.java
55         SimpleMetricsRegistry.java
56
57     --- services
58         | --- routing
59             LoggingRouter.java
60             MeteredRouter.java
61             Route.java
62             Router.java
63             RouteRequest.java
64             ThirdPartyPathfinder.java
65             ThirdPartyRouterAdapter.java
66             TimeOptimalRouter.java
67
68         | --- scenario
69             ScenarioBuilder.java
70             ScenarioLoader.java
71
72     --- themes
73         CityPackFactory.java
74         DefaultPackFactory.java
75         Palette.java
76         SofiaPackFactory.java
77
78     --- ui
79         | InspectorPanel.java
80         | MainWindow.java
81         | MainWindowMediator.java
82         | MapPanel.java
83         | TimelinePanel.java
84
85         | --- commands
86             Command.java
87             CommandStack.java
88
89     --- util
90         CancellationToken.java
91
92 --- resources
93     | --- META-INF
94     |     | --- services
95     |         metrosim.themes.CityPackFactory

```

```
96 |  
97 | --- scenarios  
98 | sample.scn
```

Listing 14: Структура на директориите

11 Бъдещо развитие и разширения

11.1 Планирани функции

- Редактор на сценарии – графичен интерфейс за създаване на сценарии
- Записване и възпроизвеждане – запазване на симулации
- Статистики в реално време – графично представяне на метриките
- Мрежови режим – симулация на множество клиенти
- 3D визуализация – по-реалистично представяне

11.2 Възможни разширения

11.2.1 Нови типове превозни средства

- Автобуси с различни капацитети
- Трамваи с фиксирани релси
- Товарни превозни средства

11.2.2 Сложни маршрути

- Условно маршрутизиране
- Динамично пренасочване
- Оптимизация по няколко критерия

11.2.3 Интерактивност

- Добавяне на превозни средства по време на симулация
- Промяна на маршрути в реално време
- Симулиране на инциденти и закъснения

12 Заключение

12.1 Постигнати цели

MetroSim успешно демонстрира:

- Практическо приложение на девет различни Design Patterns
- Безопасна паралелност в Java Swing приложения
- Модулна архитектура с ясно разделение на отговорностите
- Разширяема система за плъгини и теми
- Реално време симулация с отзивчив потребителски интерфейс

12.2 Образователна стойност

Проектът предоставя отлична основа за обучение по:

- Object-Oriented Design – правилно използване на наследяване и полиморфизъм
- Concurrent Programming – безопасна работа с множество нишки

- Software Architecture – изграждане на сложни, поддържаеми системи
- Design Patterns – практическо приложение в реален проект

12.3 Техническо качество

Кодовата база поддържа високи стандарти за качество:

- Четим и добре документиран код
- Consistent code style чрез автоматично форматиране
- Модулна структура за лесно разширение
- Ефективно използване на паметта чрез immutable обекти
- Thread-safe архитектура без състезателни условия

12.4 Практическа приложимост

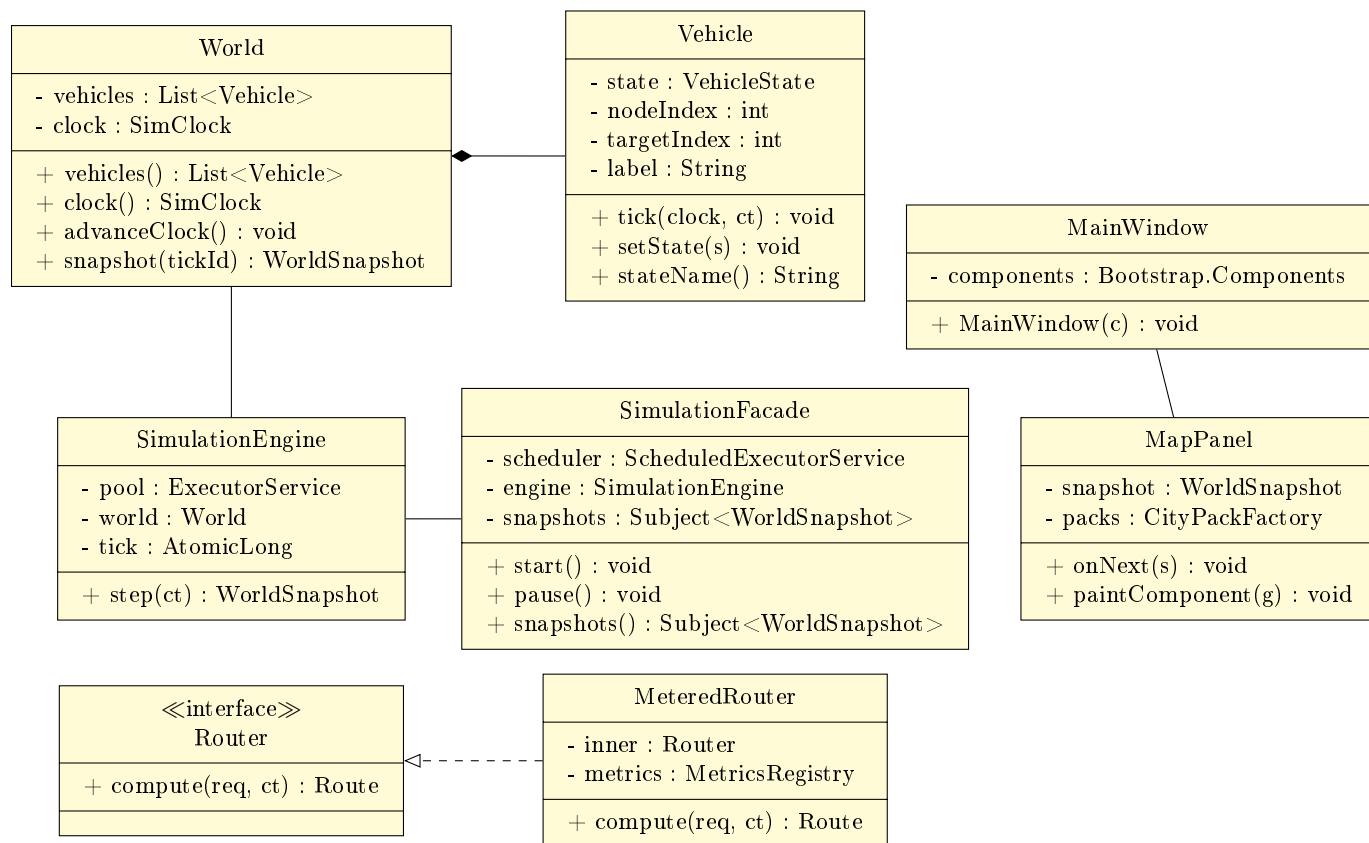
Принципите и шаблоните, демонстрирани в MetroSim, могат да се приложат в:

- Реални симулационни системи
- Игрови двигатели
- Финансови системи за трейдинг
- IoT и embedded системи
- Микросервизна архитектура

MetroSim представлява солиден пример за това как правилното прилагане на Design Patterns и принципите за добра софтуерна архитектура могат да доведат до създаването на мащабируеми, поддържаеми и разширяеми системи.

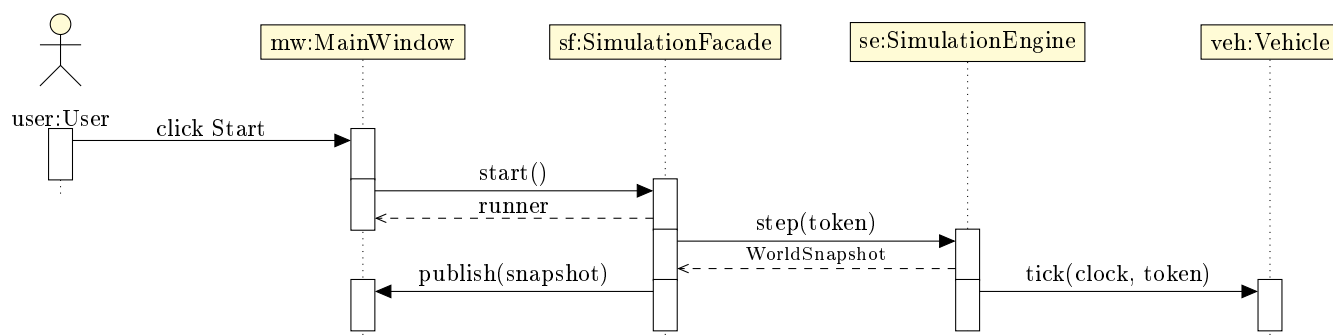
Приложения

А. Пълен UML клас диаграм на системата



Фигура 11: Опростен UML клас диаграм на цялата система

В. Последователност диаграм на симулационния цикъл



Фигура 12: Последователна диаграма на симулационния цикъл

С. Справочник на ключовите класове

Клас	Пакет	Отговорност
Bootstrap	app	Инициализация и конфигуриране на компонентите
MetroSimApp	app	Главна точка на стартиране
World	domain	Централно състояние на симулацията
Vehicle	domain	Превозно средство с вътрешни състояния
SimulationEngine	infrastructure	Ядро на симулацията с паралелна обработка
SimulationFacade	infrastructure	Опростен интерфейс за управление на симулацията
Router	services.routing	Стратегия за изчисляване на маршрути
CityPackFactory	themes	Фабрика за създаване на визуални теми
Subject	events	Събитийна система за Observer pattern
MainWindow	ui	Главен прозорец на приложението
CommandStack	ui.commands	Система за undo/redo операции

Таблица 5: Справочник на ключовите класове